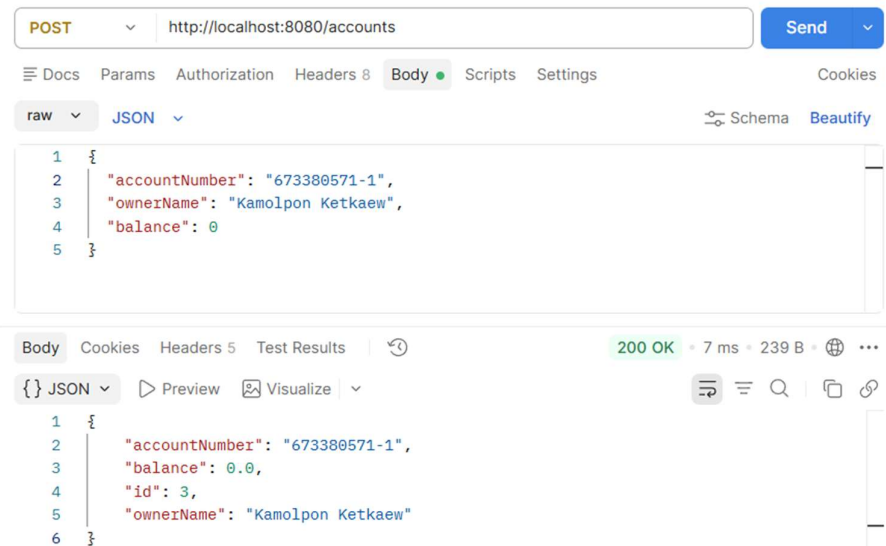


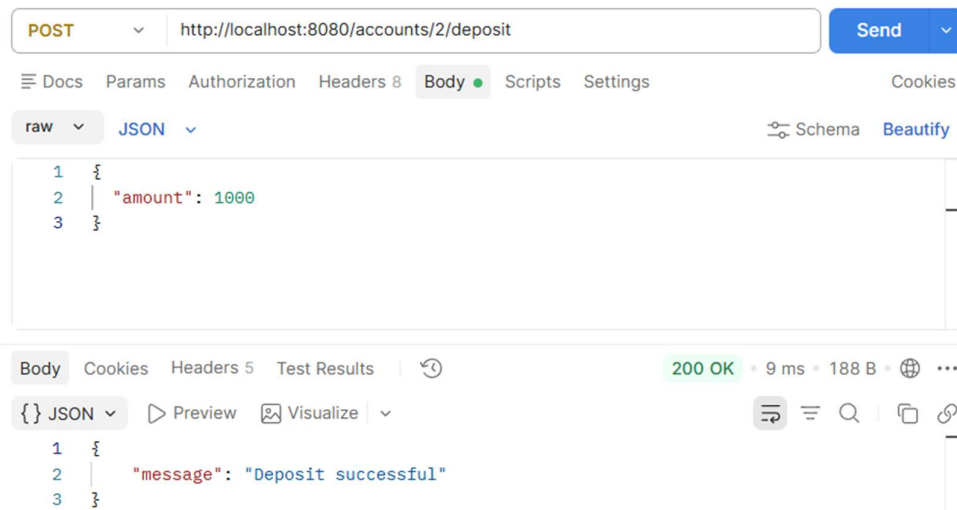
Lab 9 Report

1. การสร้าง Account



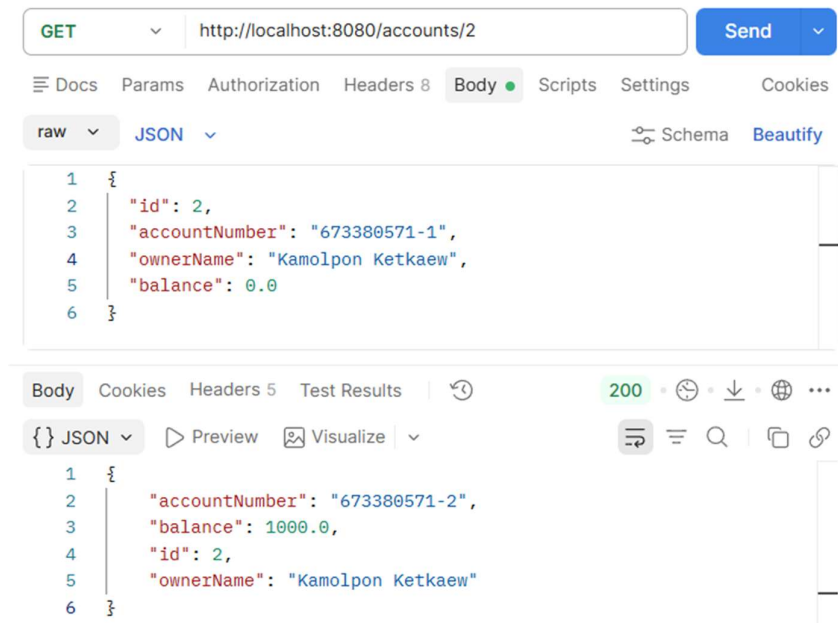
ส่งคำขอ POST /accounts ผ่าน Postman โดยระบุเลขบัญชีเป็นรหัสนักศึกษา ชื่อเจ้าของบัญชี และยอดเงินเริ่มต้น ระบบบันทึกข้อมูลบัญชีลงฐานข้อมูลและส่งข้อมูลกลับมาพร้อมรหัสบัญชี id ซึ่งใช้สำหรับอ้างอิงในการฝากเงินและตรวจสอบยอดเงินในขั้นตอนต่อไป

2. การฝากเงินสำเร็จ



ส่งคำขอ POST /accounts/2/deposit พร้อมจำนวนเงิน 1,000 บาท ระบบตอบกลับสถานะ 200 OK และข้อความ Deposit successful แสดงว่าการฝากเงินสำเร็จ โดยระบบเพิ่มยอดเงินในบัญชีและบันทึกประวัติการฝากเงินภายใน Transaction เดียวกัน

3. ตรวจสอบ Account



ส่งคำขอ GET /accounts/2 เพื่อเรียกดูข้อมูลบัญชีหลังฝากเงิน พบว่าค่า balance เพิ่มขึ้น 1,000 บาทจากยอดเดิม แสดงว่าระบบปรับปรุงยอดเงินของบัญชีที่ฝากได้ถูกต้อง

4. ตรวจสอบ DepositTransaction

Query

Query History

1

SELECT id, account_number, owner_name, balance

2

FROM account

3

ORDER BY id;

4

Data Output

Messages

Notifications

SQL

Showing rows: 1 to 3

Page No: 1 of 1

	id [PK] bigint	account_number character varying (255)	owner_name character varying (255)	balance double precision
1	1	1234567890	Kamolpon571-1	1000
2	2	673380571-2	Kamolpon Ketkaew	1000
3	3	673380571-1	Kamolpon Ketkaew	0

ตรวจสอบตาราง deposit_transaction ผ่าน pgAdmin โดยเลือกข้อมูลที่มี account_id = 2 พบประวัติการฝากเงินจำนวน 1,000 บาท ซึ่งเชื่อมโยงกับบัญชีที่ใช้ทดลองผ่าน Foreign Key account_id แสดงว่าระบบบันทึกประวัติการฝากเงินได้ถูกต้อง

5. ทดลอง Rollback

The left screenshot shows a POST request to `http://localhost:8080/accounts/2/deposit` with a JSON body `{ "amount": 1000 }`. The response is a 500 Internal Server Error with a JSON body: `{ "timestamp": "2026-09-18T13:25:55.016Z", "status": 500, "error": "Internal Server Error", "path": "/accounts/2/deposit" }`.

The right screenshot shows a GET request to `http://localhost:8080/accounts/2` with a JSON body: `{ "id": 2, "accountNumber": "673380571-1", "ownerName": "Kamolpon Ketkaew", "balance": 0.0 }`.

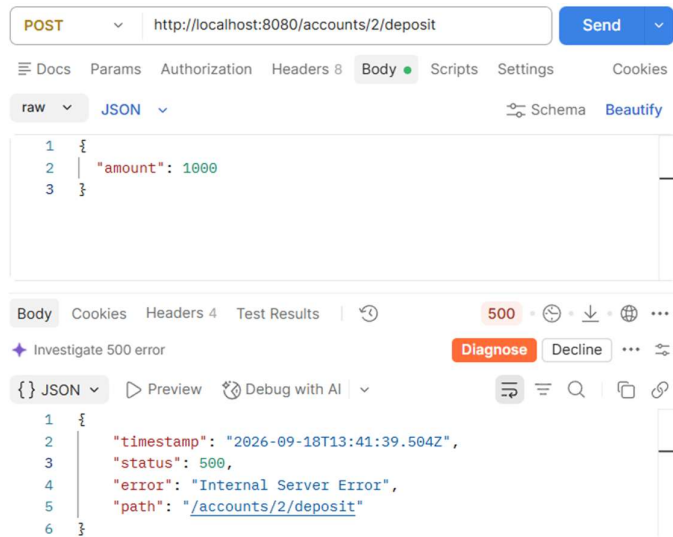
เพิ่มคำสั่ง `throw new RuntimeException("Test Rollback");` ไว้หลังคำสั่งบันทึกบัญชีและประวัติการฝากเงิน โดยยังใช้ `@Transactional` เมื่อส่งคำขอฝากเงิน ระบบจึงตอบกลับสถานะ 500 Internal Server Error จากข้อผิดพลาดที่เพิ่งสร้างขึ้น หลังเกิดข้อผิดพลาด เรียกดูข้อมูลบัญชีอีกครั้ง พบว่ายอดเงินยังเท่ากับก่อนทดลอง แสดงว่าการเพิ่มยอดเงินในการฝากครั้งนี้ถูก Rollback

The screenshot shows a database query tool interface. The query is `SELECT * FROM deposit_transaction WHERE account_id = 2 ORDER BY id;`. The results table shows one row with `id` 2, `amount` 1000, and `account_id` 2.

	id [PK] bigint	amount double precision	account_id bigint
1	2	1000	2

ตรวจสอบประวัติการฝากเงินในฐานข้อมูล พบว่าจำนวนรายการยังเท่าเดิม ไม่มีรายการใหม่จากการฝากที่เกิดข้อผิดพลาด แสดงว่า Transaction ย้อนกลับทั้งการเพิ่มยอดเงินและการบันทึกประวัติ โดยข้อมูลที่บันทึกสำเร็จก่อนหน้านี้ยังคงอยู่

6. เปรียบเทียบเมื่อไม่มี @Transactional



นำ @Transactional ออกจากเมธอด deposit() แต่ยังคงคำสั่งโยน RuntimeException ไว้
ตอนท้าย จากนั้นรันโปรแกรมใหม่และทดลองฝากเงิน ระบบตอบกลับสถานะ 500 Internal Server Error
เช่นเดียวกับการทดลองก่อนหน้านี้

Query		Query History	
1	SELECT *		
2	FROM deposit_transaction		
3	WHERE account_id = 2		
4	ORDER BY id;		

Data Output		Messages		Notifications	
Showing rows: 1 to 2		Page No: 1	of 1		
	id [PK] bigint	amount double precision	account_id bigint		
1	2	1000	2		
2	4	1000	2		

เมื่อตรวจสอบฐานข้อมูล พบว่ายอดเงินเพิ่มขึ้น 1,000 บาท และมีประวัติการฝากเงินเพิ่มหนึ่งรายการ
แม้คำขอจะตอบกลับเป็นข้อผิดพลาด เนื่องจากการเรียก save() แต่ครั้งบันทึกข้อมูลสำเร็จใน Transaction
ของตนเองก่อนเกิดข้อผิดพลาดตอนท้าย

เมื่อมี @Transactional การเพิ่มยอดเงินและการบันทึกประวัติอยู่ใน Transaction เดียวกัน จึงถูก
ย้อนกลับพร้อมกันเมื่อเกิด RuntimeException แต่เมื่อไม่มี @Transactional ครอบเมธอด ข้อมูลที่บันทึก
สำเร็จไปแล้วจะยังคงอยู่ แม้การทำงานส่วนท้ายเกิดข้อผิดพลาด

คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ตอบ @Transactional ใช้กำหนดขอบเขต Transaction เพื่อรวมการทำงานกับฐานข้อมูลหลายชั้นตอนให้เป็นงานเดียวกัน เมื่อทำงานสำเร็จจะ Commit ข้อมูล แต่หากเกิดข้อผิดพลาดที่เข้าเงื่อนไข Rollback จะยกเลิกการเปลี่ยนแปลงของ Transaction นั้น โดยค่าเริ่มต้นจะ Rollback เมื่อเกิด RuntimeException หรือ Error

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

ตอบ การฝากเงินต้องเพิ่มยอดเงินในบัญชีและบันทึกประวัติการฝากให้สำเร็จพร้อมกัน หากสำเร็จเพียงชั้นตอนเดียว ข้อมูลจะไม่สอดคล้องกัน เช่น ยอดเงินเพิ่มแต่ไม่มีประวัติ Transaction จึงช่วยให้ทั้งสองชั้นตอนสำเร็จร่วมกันหรือถูกยกเลิกร่วมกันเมื่อเกิดข้อผิดพลาด

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

ตอบ COMMIT คือการยืนยันการเปลี่ยนแปลงข้อมูลของ Transaction ลงฐานข้อมูลเมื่อทำงานสำเร็จ ส่วน ROLLBACK คือการยกเลิกการเปลี่ยนแปลงของ Transaction นั้น โดยไม่ยกเลิกข้อมูลจาก Transaction ก่อนหน้าที่ Commit สำเร็จแล้ว

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

ตอบ มีความสัมพันธ์แบบ One-to-Many คือหนึ่งบัญชีสามารถมีประวัติการฝากเงินได้หลายรายการ ส่วนประวัติแต่ละรายการอ้างอิงบัญชีเดียว ในแลบนี้กำหนด @ManyToOne ที่ฝั่ง DepositTransaction และใช้คอลัมน์ account_id เป็น Foreign Key อ้างอิงไปยัง id ของ Account

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ตอบ ในการทดลอง เมื่อเกิด RuntimeException และข้อผิดพลาดถูกส่งออกจากเมธอด ระบบจะ Rollback การเปลี่ยนแปลงทั้งหมดของการฝากครั้งนั้น ทำให้ยอดเงินและจำนวนประวัติการฝากคงเดิม ส่วนข้อมูลจากการฝากที่สำเร็จก่อนหน้านี้ยังคงอยู่ในฐานข้อมูล

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

ตอบ เมื่อมี @Transactional ข้อผิดพลาดตอนท้ายทำให้ยอดเงินและประวัติของการฝากครั้งนั้นถูกย้อนกลับ แต่เมื่อนำออก คำสั่ง save() แต่ละครั้งบันทึกข้อมูลเสร็จแยกกัน จึงพบว่ายอดเงินและประวัติเพิ่มขึ้น แม้ระบบตอบสถานะ 500 แสดงว่าไม่มี Transaction เดียวครอบคลุมการฝากเงินทั้งหมด